PrintWatch

Monitoreo de impresión 3D con YOLO11

Detección de defectos capa por capa con visión por computador

Samuel Esteban Rodríguez Marín · Adrian Stiven Meneses Ávila
Docente: Jheyston Serrano
Ingeniería en Inteligencia Artificial · UIS Barrancabermeja · 2026

#LaUISqueQueremos

Del píxel a la detección

1



Captura

1280 × 720

2



Preproceso

640 × 640 · [0, 1]

3



Red YOLO11s

una sola pasada

4



Filtrado

umbral + NMS

5



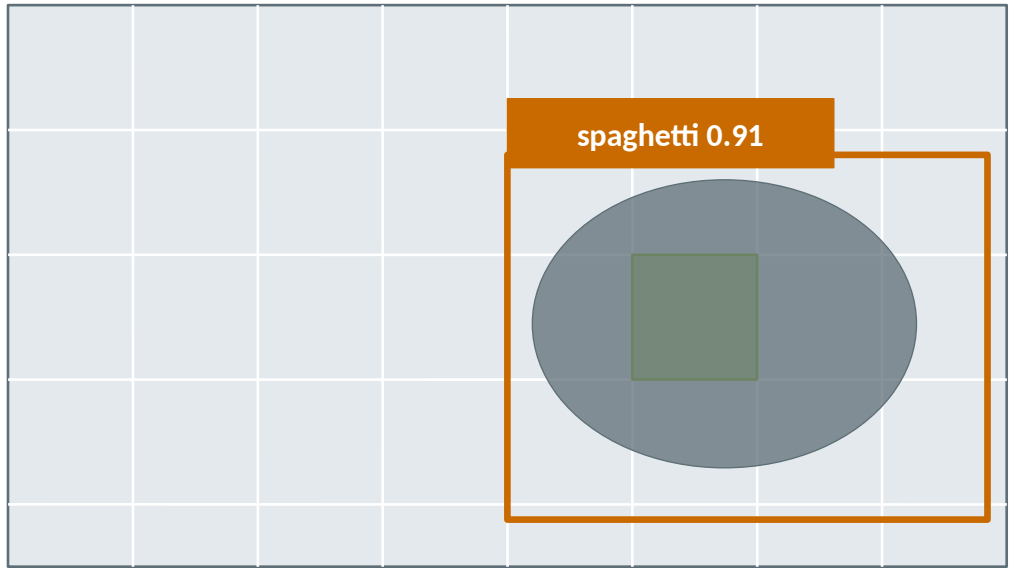
Resultado

clase + caja

Una foto por capa

Modelo FP32 en ONNX Runtime, sobre la CPU de un laptop, sin GPU dedicada.

¿Cómo ve YOLO? Una sola pasada



Grilla ilustrativa: la real es 80×80, 40×40 y 20×20

YOLO: imagen → red → cajas y clases (1 paso)

Faster R-CNN: proponer regiones → clasificar (2 etapas)

1

Una sola pasada

Localiza y clasifica a la vez, sin proponer regiones.

2

Celdas en 3 escalas

$80^2 + 40^2 + 20^2 = 8\,400$ candidatas; cada una predice caja + 6 clases.

3

NMS

Elimina duplicados: queda una caja por defecto.

#LaUISqueQueremos

Arquitectura de YOLO11s



≈ 9.4 M
parámetros

≈ 21.5
GFLOPs por imagen

6
clases de defecto

Cifras oficiales de Ultralytics a 640×640.

Backbone y Neck

C3k2 (CSPDarknet)

Divide el flujo en dos ramas y las une: menos cómputo con la misma capacidad.

SPPF

Pooling en cascada: ve defectos pequeños y grandes con el mismo costo.

C2PSA

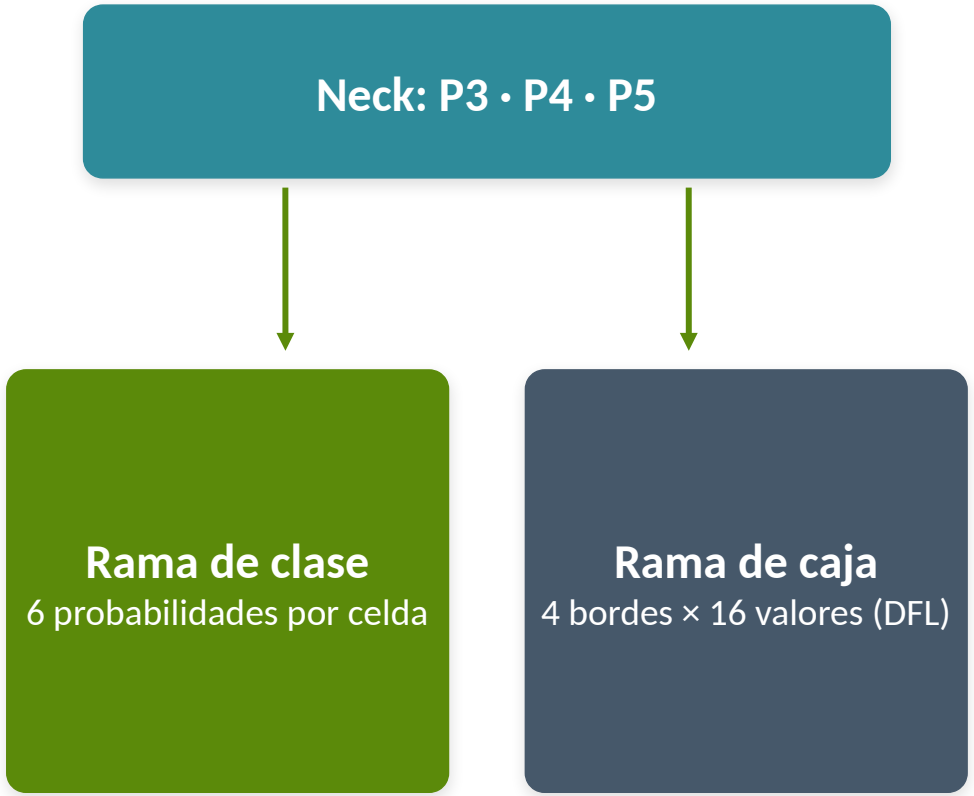
Atención espacial: pondera qué regiones de la imagen importan más.

PAN-FPN: P3, P4, P5

Strides 8, 16 y 32. P3 aporta detalle fino y P5 contexto; se fusionan en dos sentidos.

#LaUISqueQueremos

Head: sin anchors y desacoplada



Mismo mapa de entrada, pesos distintos

Anchor-free

Cada celda predice las distancias a los 4 bordes. Sin cajas de referencia.

Desacoplada

Clasificar y localizar son tareas distintas: separarlas mejora ambas.

DFL (reg_max = 16)

Cada borde es una distribución sobre 16 valores

valor = Σ i · P(i), i = 0…15

Preprocesamiento: letterbox

1 **Captura cruda**

1280 × 720

3 canales · enteros 0 a 255

2 **Escala sin deformar**

640 × 360

s = min(640/w, 640/h) = 0.5

3 **Relleno gris 114**

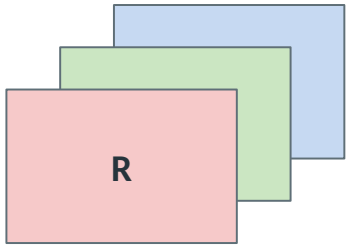


140 px de relleno arriba y abajo

En producción el worker redimensiona directo (sx = 0.5, sy ≈ 0.889), sin letterbox. En 13 fotos reales ninguno fue mejor; queda como limitación abierta.

Preprocesamiento: tensor y escala

4 HWC → CHW + batch



(1, 3, 640, 640)

lote · canales · alto · ancho

5 Normalización

$$x' = x / 255$$

[0, 255] → [0, 1]

¿Por qué no se resta media/std?

La primera capa de BatchNorm ya recentra y reescala la entrada. Es la normalización estándar de Ultralytics.

¿Por qué 640?

640 = 32 × 20

El backbone reduce 5 veces por 2 ($2^5 = 32$), y es el tamaño de preentrenamiento.

#LaUISqueQueremos

De la salida cruda a las cajas

8 400 candidatas
tensor (1, 4 + 6, 8400)

↓

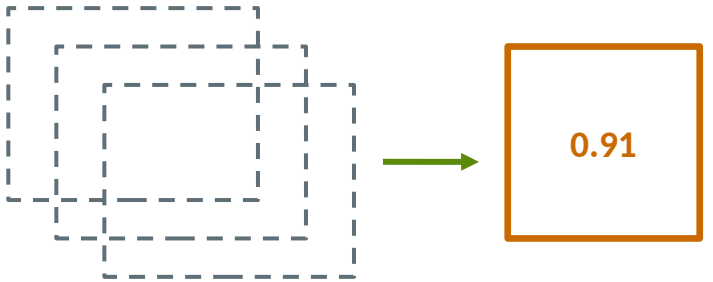
Umbral de confianza 0.25
quedan decenas o menos

↓

NMS · IoU > 0.45
una caja por defecto

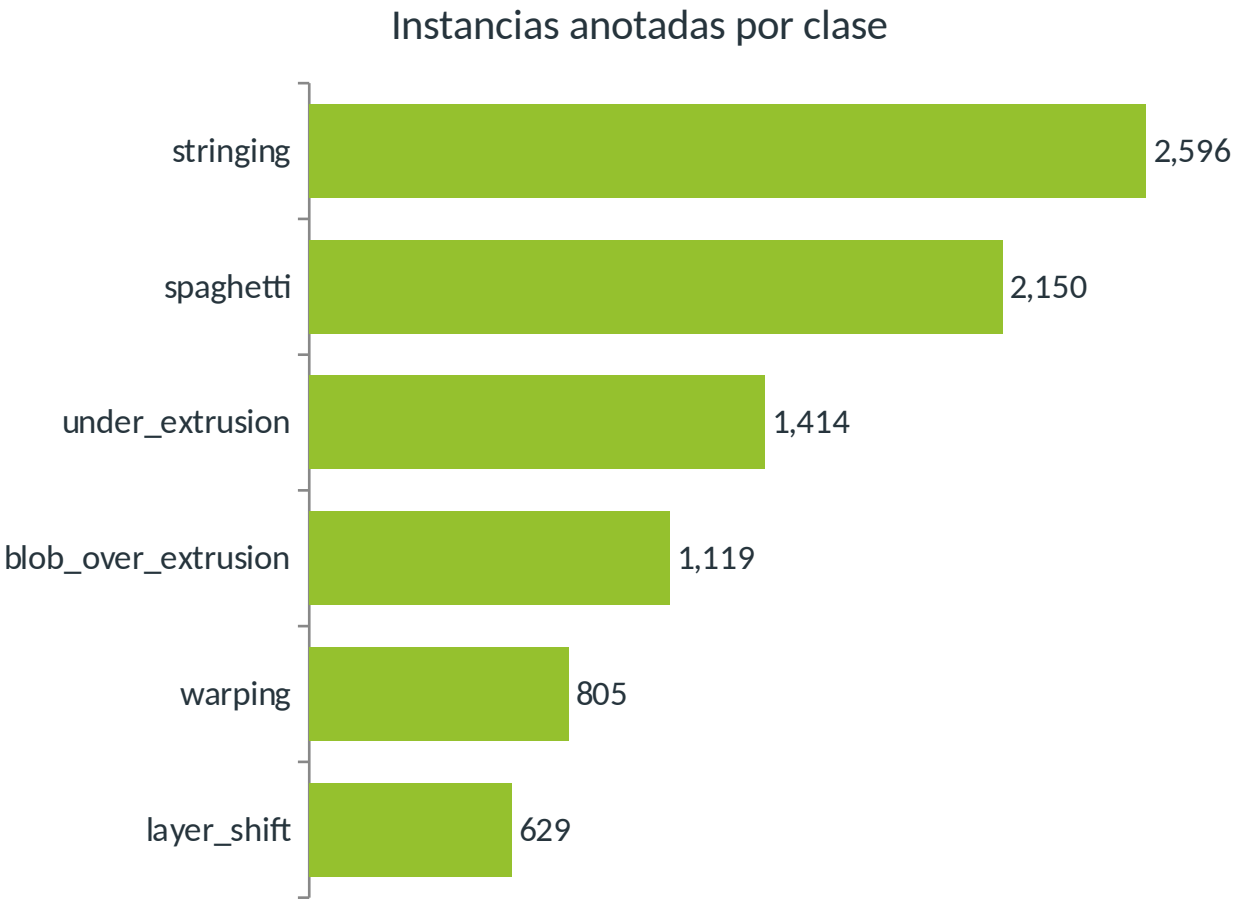
NMS agnóstico a la clase: dos clases no sobreviven sobre el mismo objeto.

Ejemplo de NMS



Sin NMS: 3 cajas casi iguales para un solo defecto. Con NMS: la de mayor confianza.

Dataset: 5 613 imágenes



Fuentes (Roboflow Universe)

3 612 — 3D PRINT DEFECT DETECTION

1 002 — 3d print fail detection

999 — Under Extrusion

Se descartaron clases sin equivalente, como crack.

#LaUISqueQueremos

División 70 / 15 / 15 sin fuga

Entrenamiento · 70 %	Val · 15 %	Test · 15 %
≈ 3 929 imágenes	≈ 842	841 (real)

Total 5 613 · semilla 42 · estratificado de forma aproximada. El test real es de 841 porque la división por secuencia no es exacta.

× Por fotograma, al azar



✓ Por secuencia de video



Train Val Test

Frames vecinos en dos conjuntos inflan las métricas (data leakage).

Por eso las métricas de prueba son honestas: el modelo nunca vio esas secuencias.

#LaUISqueQueremos

Entrenamiento

YOLO11s
ganó el bake-off

100
épocas máximo

16
tamaño de lote

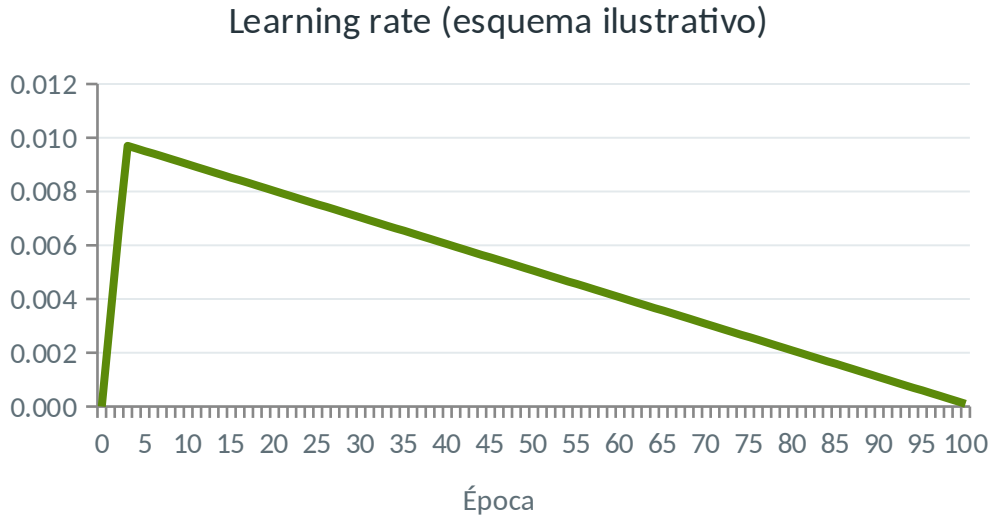
640 px
tamaño de imagen

20
patience (parada)

0.01
lr inicial (lr_0)

auto
optimizador Ultralytics

42
semilla · Colab GPU



Warmup al inicio, luego decaimiento hasta ~1 % del lr_0.
Regla base: $v \leftarrow \beta v + (1-\beta)\nabla L$; $w \leftarrow w - \eta(v + \lambda w)$.
Aumento (solo train): mosaic, HSV, flip.

Función de pérdida

$$L = 7.5 \cdot L_{CIoU} + 0.5 \cdot L_{BCE} + 1.5 \cdot L_{DFL}$$

7.5

CIoU · caja

Superposición + distancia entre centros + proporción. Da gradiente aunque las cajas no se toquen.

0.5

BCE · clase

Entropía cruzada binaria por clase y por celda: ¿es spaghetti o no?

1.5

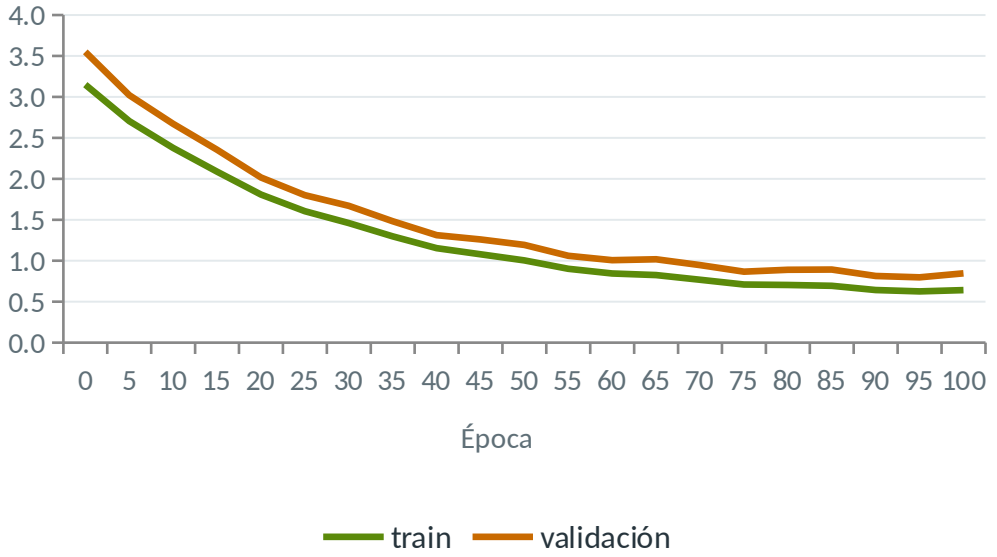
DFL · bordes

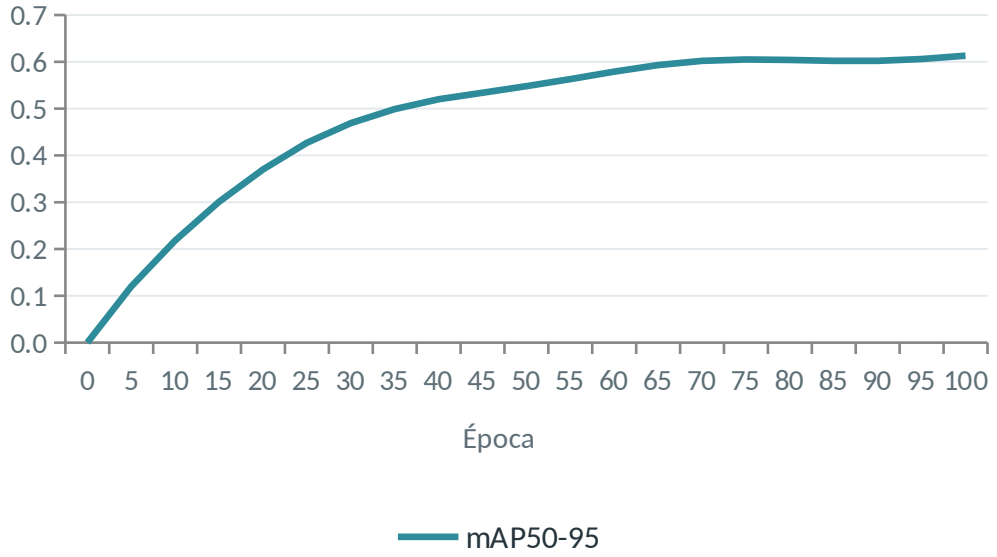
Afina cada borde como una distribución sobre 16 valores.

La caja pesa más porque localizar con precisión es más difícil que clasificar.

#LaUISqueQueremos

Curvas de entrenamiento





Sano

Train y validación bajan juntas.

Overfitting

Validación se aplana o sube; la brecha crece. Patience corta ahí.

Checkpoint

Se conserva el mejor, no el de la última época.

#LaUISqueQueremos

Resultados y límites del modelo

Clase	Precisión	Recall
General (¿hay defecto?)	0.997	0.921
spaghetti	0.963	0.975
warping	0.988	0.988
stringing	0.977	0.959
layer_shift	1.000	1.000
under_extrusion	**0.919**	**0.685**
blob_over_extrusion	**0.562**	**0.529**

841 imágenes de prueba · umbral 0.25 · nivel imagen

Fotos de Roboflow

No se midió con nuestra cámara ni con spaghetti real.

Enredo pequeño

Ocupa 0.22 % de la imagen y no se detecta; el grande pasa por 0.07.

Clases débiles

under_extrusion pierde 1 de cada 3 y blob_over_extrusion solo informa.